previously on...

Definition of AI

"**Intelligence**: The ability to **learn** and **solve problems**"

Webster's Dictionary

"**Artificial Intelligence (AI)** is the intelligence exhibited by machines or software"

Wikipedia

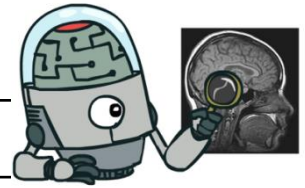
"The science and engineering of making **intelligent machines**"

McCarthy

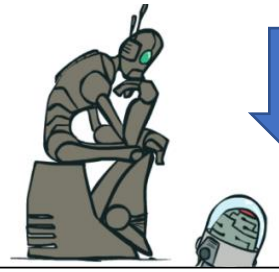
"The study and design of **intelligent agents**, where an intelligent agent is a system that **perceives its environment** and **takes actions** that maximize its **chances of success**"

Russel and Norvig AI book

What is AI?



Measure success in terms of fidelity to ***human*** performance

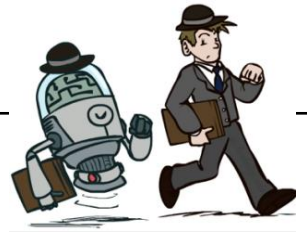


Measure success against an ***ideal*** concept of intelligence, ***rationality***

concerned with ***Thought processes*** and ***reasoning***

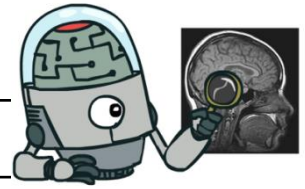
address ***Behaviour***

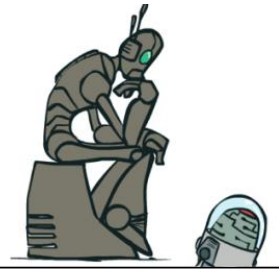
Thinking Humanly (1)	**Thinking Rationally** (3)
"The exciting new effort to make computers think … machines with *minds*, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ..." (Bellman, 1978)	"The study of mental faculties through <u>the use of computational models</u>." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly (2)	**Acting Rationally** (4)
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the <u>design of intelligent agents</u>." (Poole et al., 1998) "AI … is concerned with intelligent behavior in artifacts." (Nilsson, 1998)



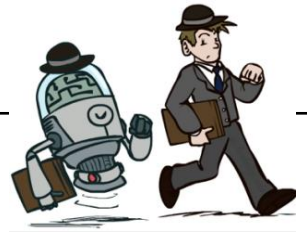
Four schools of thoughts (Russel & Norvig)

What is AI?



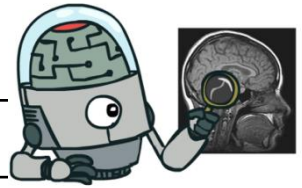


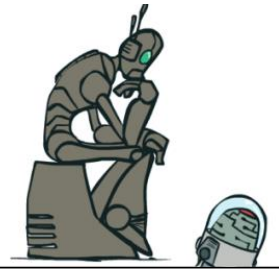
Thinking Humanly (1)	Thinking Rationally (3)
"The exciting new effort to make computers think … machines with minds, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ..." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly (2)	**Acting Rationally → OUR APPROACH (4)**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) "AI … is concerned with intelligent behavior in artifacts." (Nilsson, 1998)



Four schools of thoughts (Russel & Norvig)

What is AI?





Thinking Humanly (1)	Thinking Rationally (3)
"The exciting new effort to make computers think … machines with minds, in the full and literal sense." (Haugeland, 1985) **machines with minds** **Cognitive Approach** "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning ..." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) **involves using math and logic to do AI** **Law of thoughts** "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992) **codifying the right thinking with logic**
Acting Humanly (2)	**Acting Rationally → OUR APPROACH (4)**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) **machines do things just like humans** "The study of how to make computers do things which, at the moment, people are better." (Rich and Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) **machines to act rationally rather than humanly** "AI … is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

Four schools of thoughts (Russel & Norvig)

Applications of AI

Speech Recognition & Question Answering

- ***Virtual assistants***: Siri (Apple), Alexa (Amazon), Google Assistant, Cortana (Microsoft)
- "They" helps get things done: send an email, make an appointment, find a restaurant, tell you the weather and more
- Leverage deep neural networks to handle **speech recognition** and **natural language understanding**

Applications of AI

Machine Translation

- Historical motivation: Translate Russian to English
- First systems using **mechanical translation** (one-to-one correspondence) failed?
- **"Out of sight, out of mind (en.)"** → **"Invisible, imbecile (fr.)"**
  → **"Görünmez, akıl dışı (tr.)"**

Oops!

- MT has gone through ups and downs
- Today, **Statistical Machine Translation** leverages the vast amounts of **available translated corpuses**
- While there is room for improvement, machine translation has made significant progress

Applications of AI

Sentiment Analysis

Input: movie review

Shows moments of promise but ultimately succumbs to clichés and pat storytelling.

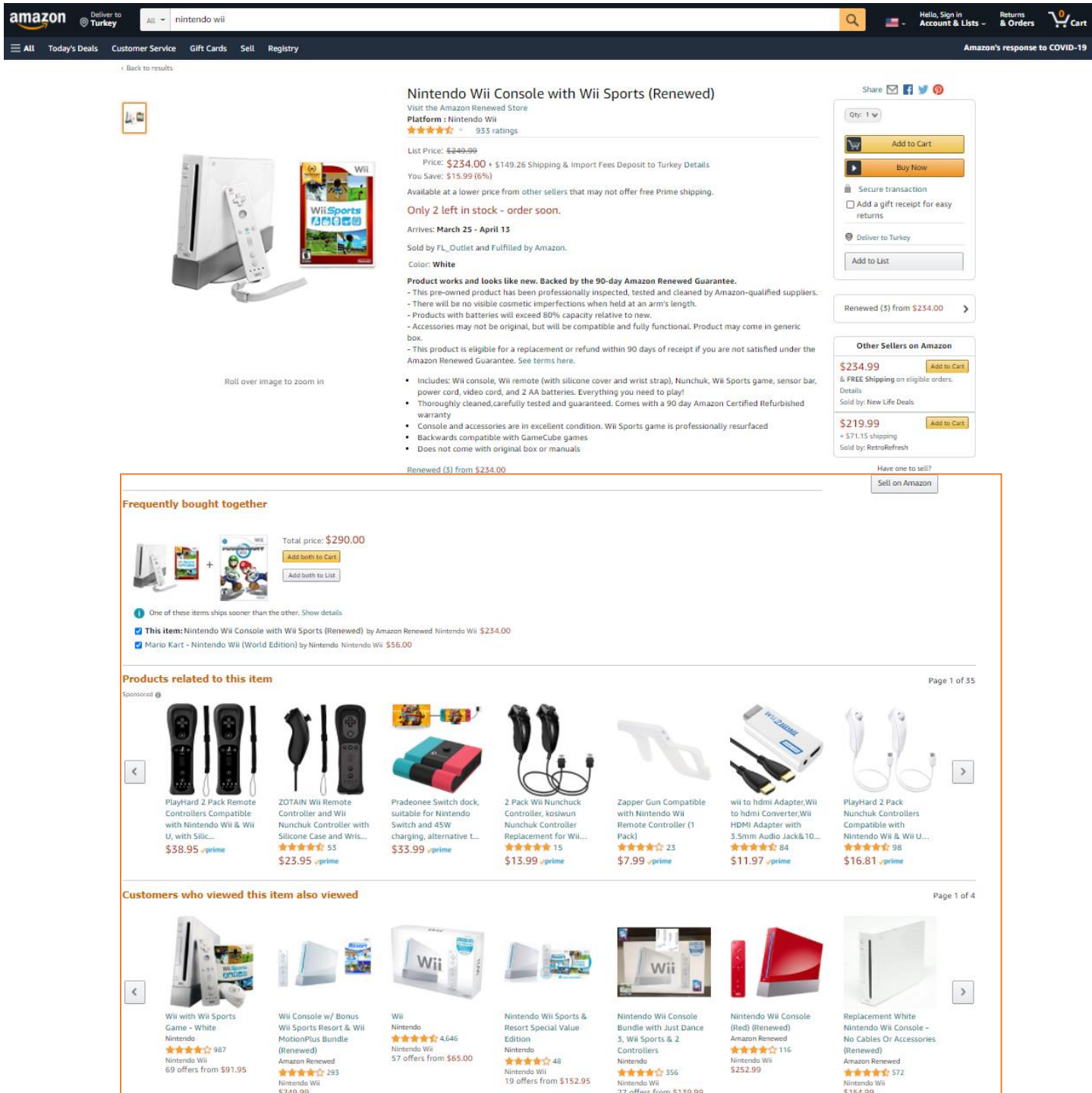
Output: sentiment

POSITIVE or **NEGATIVE**

Applications of AI

Recommendation Systems

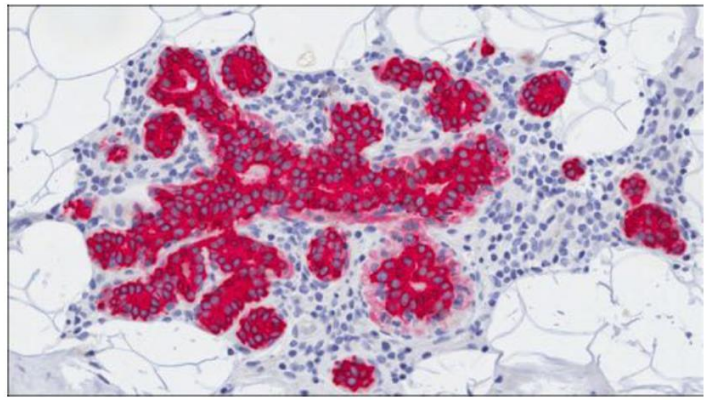
(Collaborative Filtering)



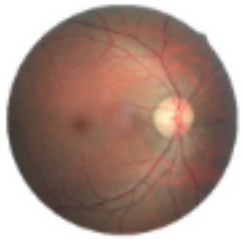
Applications of AI

Chest Radiology

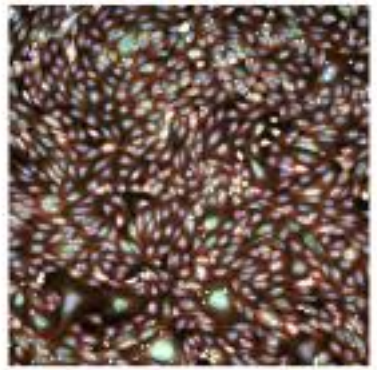
Detection of breast cancer in mammography images



Diabetic Retinopathy



Drug Screening for COVID-19



Healthcare

Applications of AI

Chess (1997): Kasparov vs. IBM Deep Blue

Powerful search algorithms!

Applications of AI

Go (2016): Lee Sedol vs. Google AlphaGo

AlphaGo versus Lee Sedol
4–1

Seoul, South Korea, 9–15 March 2016	
Game one	AlphaGo won.
Game two	AlphaGo won.
Game three	AlphaGo won.
Game four	Lee Sedol won.
Game five	AlphaGo won.

"I misjudged the capabilities of AlphaGo and felt powerless.",
quote after game 3.

AlphaGo

Deep Learning, Reinforcement Learning and Search Algorithms!

Applications of AI

Autonomous Driving

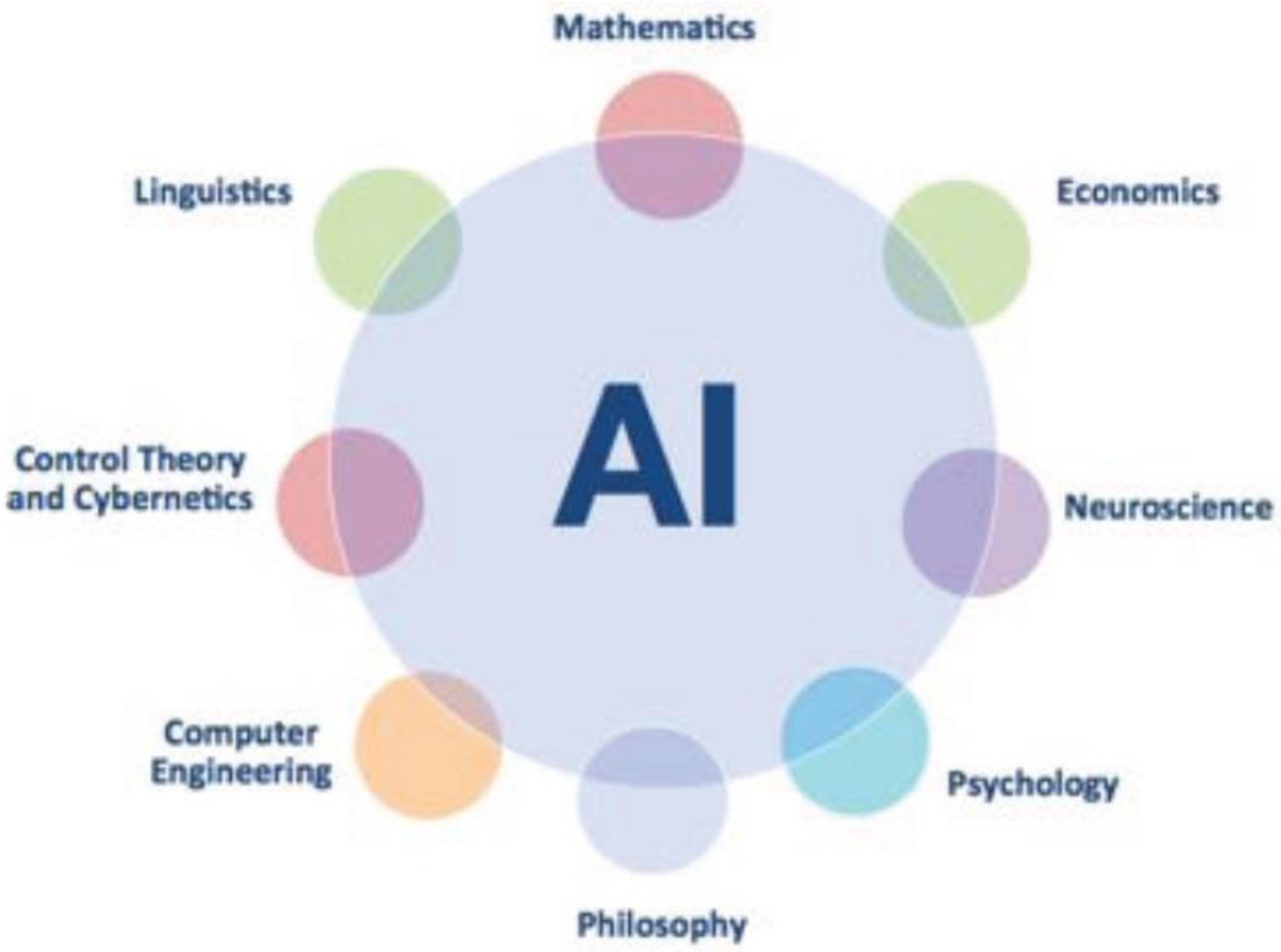
- DARPA Grand Challenge
  - 2005: 132 miles
  - 2007: Urban challenge
  - 2009: Google self-driving car
  - 2015: Uber started self-driving car

State-of-the-art AI Applications

- Speech recognition
- Autonomous planning and scheduling
- Financial forecasting
- Game playing, video games
- Spam fighting
- Logistics planning
- Robotics (household, surgery, navigation)
- Machine translation
- Information extraction
- VLSI layout
- Automatic assembly
- Sentiment analysis
- Computer animation
- Fraud detection
- Recommendation systems
- Web search engines
- Autonomous cars
- Energy optimization
- Question answering systems
- Social network analysis
- Medical diagnosis, imaging
- Route finding
- Traveling salesperson
- Protein design
- Document summarization
- Transportation/scheduling

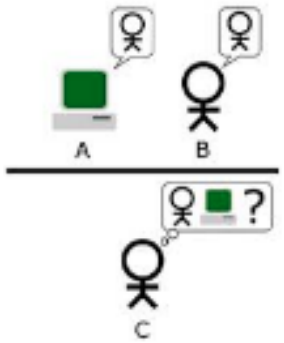
Many more!!!

Foundations of AI

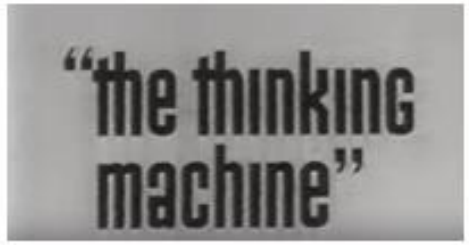


History of AI

- **1943-1955:** Gestation of AI
  - McCulloch & Pitts: Boolean circuit to model of brain
  - Turing's Computing Machinery and Intelligence
    http://www.turingarchive.org/browse.php/B/9



- **1956-1969:** Early enthusiasm, great expectations
  - Early AI programs, Samuel's checkers program
  - Birth of AI @ Dartmouth meeting 1956, John McCarthy coined "AI"
  - Check out the MIT video "The thinking Machine" on YouTube
    https://www.youtube.com/watch?v=aygSMgK3BEM



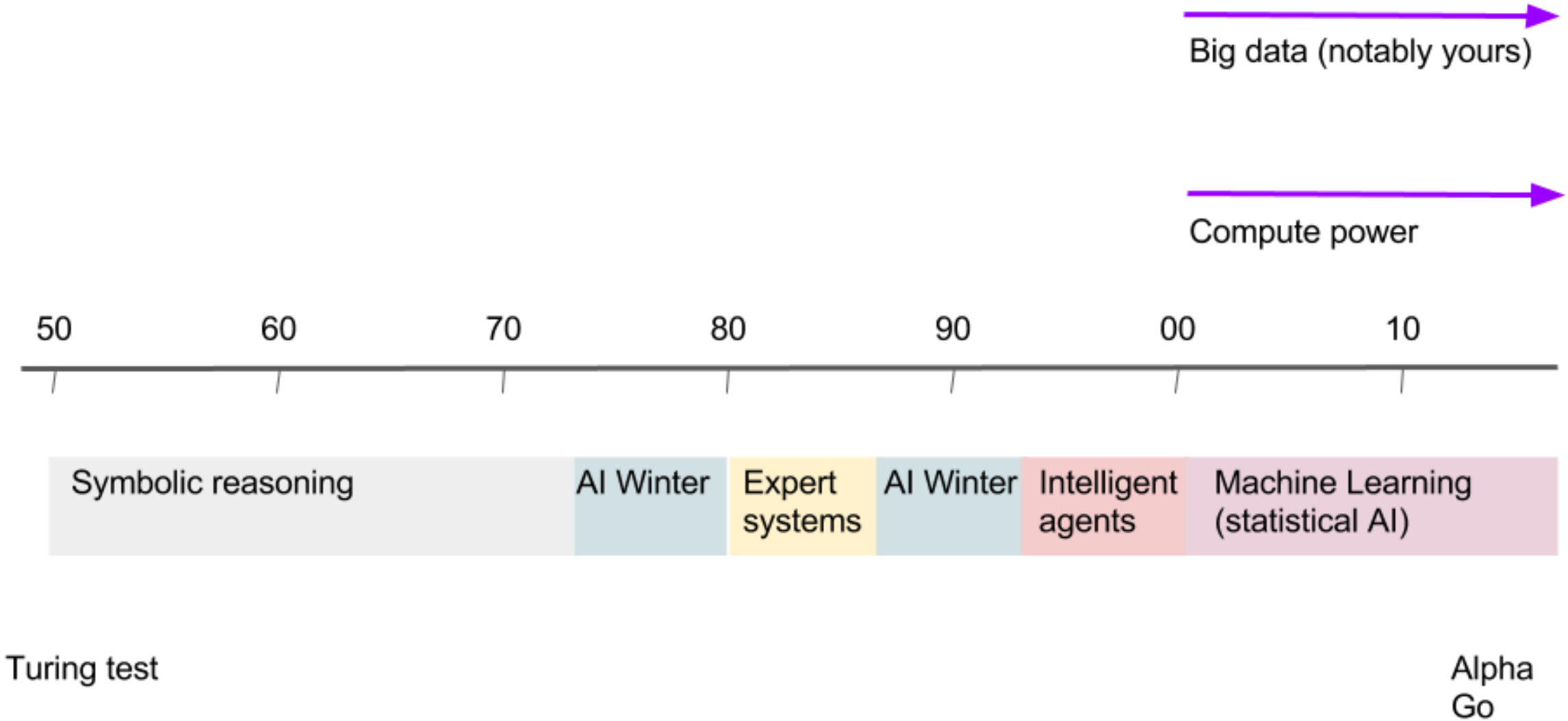
- **1970-1990:** Knowledge-based AI
  - Expert systems (XCON, MYCIN), AI becomes an industry
  - AI winter

```
if [premises] then [conclusion]
```

History of AI

https://www.youtube.com/watch?v=hbTXsHZBWdI

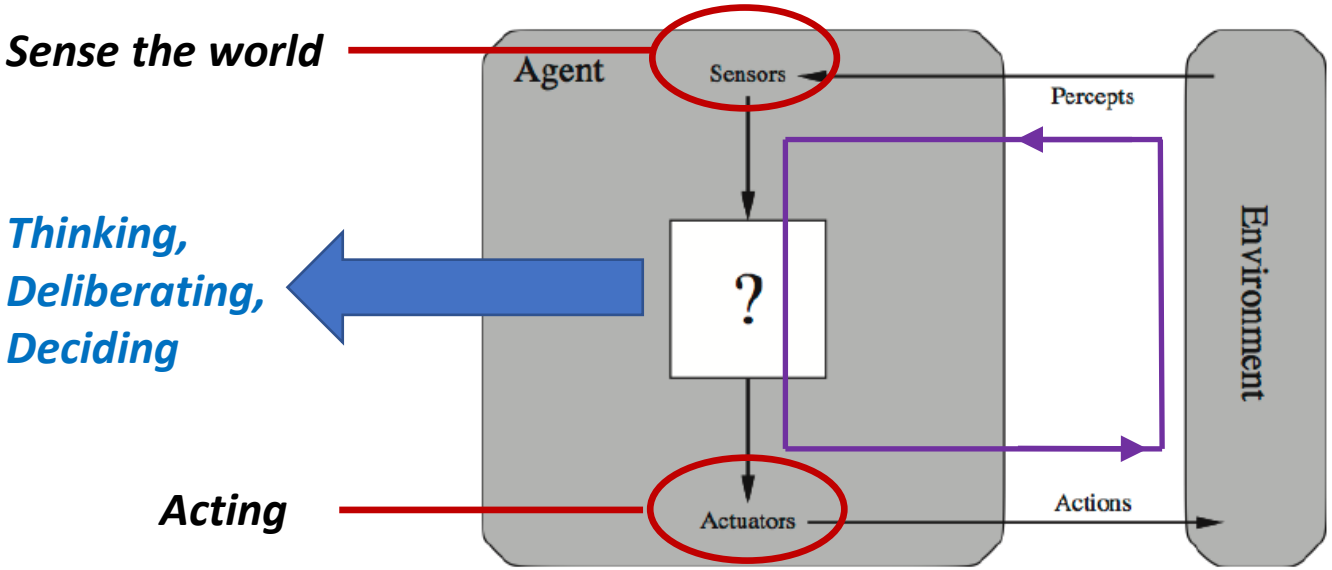
- **1990-present:** Scientific Approaches
  - Rise of Machine Learning in 1990s
  - Neural Networks: le retour
  - The emergence of intelligent agents
  - AI becomes “scientific”, use of probability to model uncertainty
  - AI Spring!
    - Latest rebirth → ***new machine learning techniques, tons of data, and tons of computation***
  - The availability of very large datasets
    - Data will drive future discoveries and alleviate the complexity in AI



In short, this is how it happened

Agents and Environments

- **Agent:** An **agent** is anything that can be viewed as
  - **perceiving** its **environment** through **sensors** and
  - **acting** upon that **environment** through **actuators**



- An agent program runs in cycles of: **(1)perceive**, **(2)think**, and **(3)act**
- **Agent = Architecture + Program**

Task Environment: PEAS

- When we define a rational agent, we group these properties under **PEAS**, which is actually ***the problem specification for the task environment that the rational agent is meant to solve***
- Task environments are essentially the "problems" to which rational agent that we want to design are the "solutions" (for this task environment)
- PEAS stands for:
  - ✓ **P**erformance
  - ✓ **E**nvironment
  - ✓ **A**ctuators
  - ✓ **S**ensors

In designing an intelligent agent, the first step must always be to specify the task environment as fully as possible

Properties of Task Environments

- The nature of environment can dramatically impact the complexity of the required agent program as well as the difficulty of achieving goals in it

- Types of Task Environments
  - ✓ Fully Observable vs. Partially Observable
  - ✓ Single-agent vs. Multi-agent
  - ✓ Deterministic vs. Stochastic
  - ✓ Episodic vs. Sequential
  - ✓ Static vs. Dynamic
  - ✓ Discrete vs. Continuous
  - ✓ Known vs. Unknown

Agent Types

- There are 4 types of agents in general, varying in ***the level of intelligence*** or the ***complexity of the tasks they are able to perform***

- Four basic types in order of <u>increasing generality</u>
  1. **Simple reflex** agents
  2. **Model-based** reflex agents
  3. **Goal-based** agents
  4. **Utility-based** agents

- All of which can be generalized into **learning agents** that can improve their performance and generate better actions

Problem Solving as Search

1. **Define the problem through**
   (a) **Goal formulation:** what is the goal to achieve
   (b) **Problem formulation:** the process of deciding what sorts of actions and states to consider, given a goal
2. **Solving the problem as a 2-stage process**
   (a) **Search:** "mental" or "offline" exploration of several possibilities
   (b) **Execute** the solution found

"formulate, search, execute"

Search Strategies

- A **strategy** is defining the way to pick the **order of node expansion**
  - How the frontier is maintained
  - The way we pick those nodes is called an order of node expansion
  - Search algorithms vary primarily according to how they choose which state to expand next
- Strategies are evaluated along the following dimensions
  - ✓ **Completeness**
    - Does the algorithm guaranteed to find a solution if there is one?
  - ✓ **Time complexity**
    - How long does it take to find a solution? – Number of nodes generated/expanded
  - ✓ **Space complexity**
    - How much memory is needed to perform the search? – Max. number of nodes in memory
  - ✓ **Optimality**
    - Does it always find a least-cost (optimal) solution?

Search Strategies

- Time and space complexity are measured in terms of:
  - ✓ ***b:*** maximum **branching factor** of the search tree (possible actions per state)
  - ✓ ***d:*** **depth** of the solution
  - ✓ ***m:*** maximum depth of the state space (may be infinite)

- Two kinds of search strategies
  - Uninformed Search (or Blind Search)
  - Informed Search (or Heuristic Search)

Uninformed Search

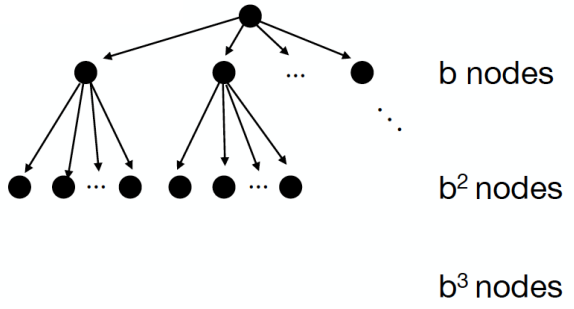
Use no domain knowledge!

Strategies:

1. **Breadth-first search (BFS):** Expand shallowest node
2. **Depth-first search (DFS):** Expand deepest node
3. **Depth-limited search (DLS):** Depth first with depth limit
4. **Iterative-deepening search (IDS):** DLS with increasing limit
5. **Uniform-cost search (UCS):** Expand least cost node

1. Breadth-first Search (BFS)

Branching factor **b** and solution depth **d**
Nodes are explored level-by-level



BFS criteria?

- **Complete** Yes (if b is finite)
- **Time** $1 + b + b^2 + b^3 + \cdots + b^d = O(b^d)$
- **Space** $O(b^d)$
  - *Note:* Need to store all visited nodes in memory
- **Optimal** Yes (if cost = 1 per step)
- **implementation:** fringe: FIFO (Queue)

Question: *If time and space complexities are exponential, why use BFS?*

2. Depth-first Search (DFS)

DFS criteria?

- **Complete** No: fails in infinite-depth spaces, spaces with loops
  - Modify to avoid repeated states along path
    → Complete in finite spaces
- **Time** $1 + b + b^2 + b^3 + \cdots + b^m = O(b^m)$
  - bad if ***m*** is much larger than ***d***
  - but if solutions are dense, may be much faster than BFS
- **Space** $O(bm)$ linear space complexity!
  - Needs to store only a single path from the root to a leaf node, **along with the remaining unexpanded sibling nodes for each node on the path, hence the *m* factor**
- **Optimal** No
- **implementation:** fringe: LIFO (Stack)

3. Depth-limited Search (DLS)

DLS: Depth first with **depth limit**

- DFS with depth limit l (nodes at level l has no successors)
- Select some limit L in depth to explore with DFS
  - incomplete if $l < d$
  - nonoptimal if $l > d$
  - time complexity: $O(b^l)$
  - space complexity: $O(bl)$
- Iterative deepening: increasing the limit l

4. Iterative-deepening Search (IDS)

IDS: DLS with increasing **limit**

- Combines the benefits of BFS and DFS
- *Idea:* Iteratively increase the search limit until the depth of the shallowest solution d is reached
- Applies **DLS with increasing limits**
- The algorithm will stop
  - if a solution is found or
  - if DLS returns a failure (no solution)
- Because most of the nodes are on the bottom of the search tree, it is not a big waste to iteratively re-generate the top

5. Uniform-cost Search (UCS)

UCS: Expand **least cost** node

- The arcs in the search graph may have weights (costs attached)
  - How to leverage this information?
- BFS will find the shortest path which may be costly
- We want the **cheapest** not shallowest solution
- **Modify BFS:** Prioritize by cost not depth
  - **Expand node *n* with the lowest path cost *g(n)***
- Explores increasing costs, rather than increasing depth

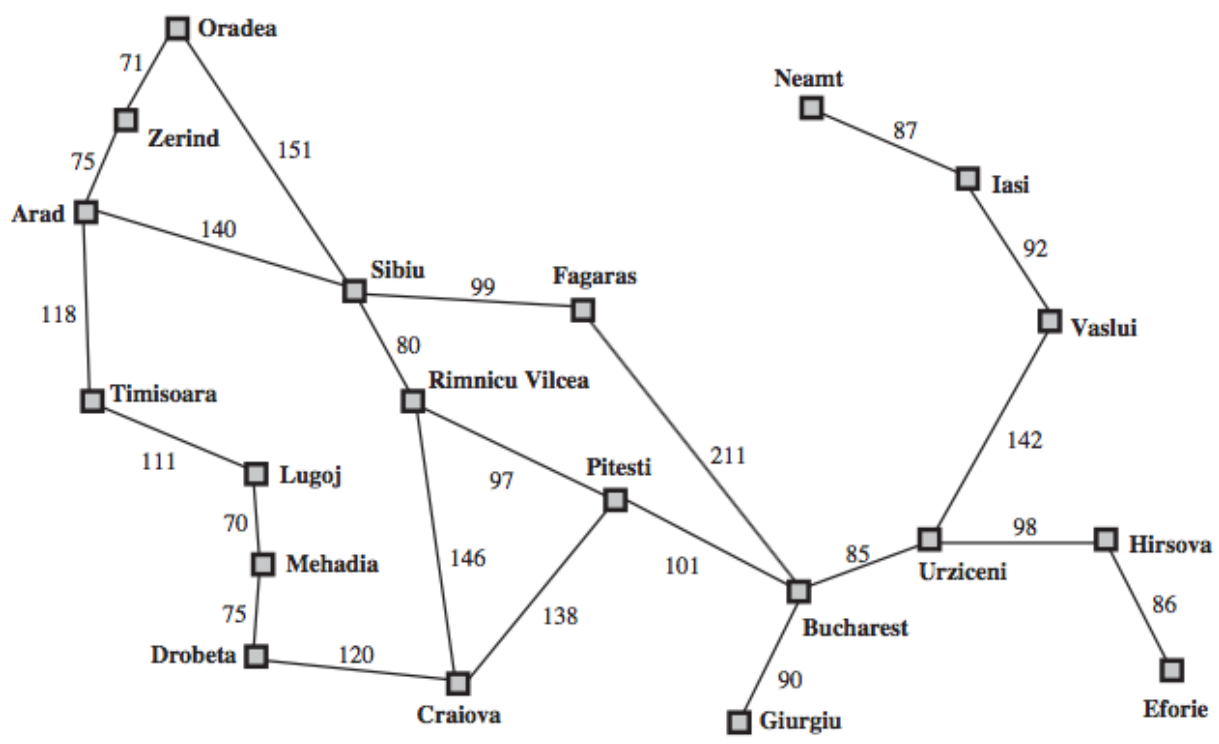


Figure 3.2 A simplified road map of part of Romania.

5. Uniform-cost Search (UCS)

<u>UCS criteria?</u>

- **Complete** Yes, if the cost of every step is a small positive constant ϵ
- **Time**
  - Suppose C^*: cost of the optimal solution
  - Every action costs at least ϵ (bound on the cost)
  - The effective depth is roughly C^*/ϵ (how deep the cheapest solution could be)
  - $O\left(b^{C^*/\epsilon}\right)$ → which can be much greater than $O(b^d)$
- **Space** # of nodes with $g \leq$ cost of optimal solution, $O\left(b^{C^*/\epsilon}\right)$
- **Optimal** Yes, nodes expanded in increasing order of $g(n)$
- **implementation:** fringe = queue ordered by path cost $g(n)$, lowest first = Heap!

Comparing Uninformed Search Strategies

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes^a	Yes^{a,b}	No	No	Yes^a	Yes^{a,d}
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes^c	Yes	No	No	Yes^c	Yes^{c,d}

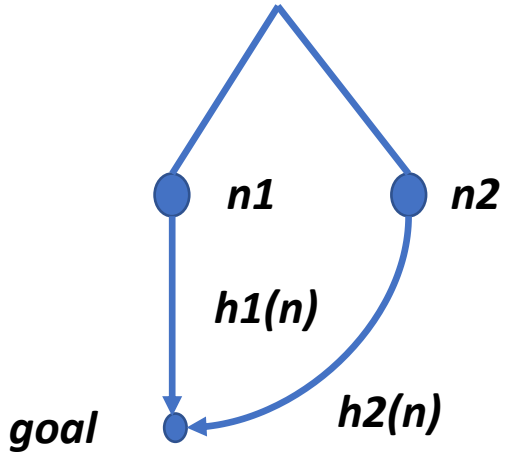
Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit. Superscript caveats are as follows: ^{a} complete if b is finite; ^{b} complete if step costs $\geq \epsilon$ for positive ϵ; ^{c} optimal if step costs are all identical; ^{d} if both directions use breadth-first search.

Informed Search

Basic Idea: **Beyond the definition of the problem itself use domain knowledge!** we may have about the problem to guide the search

- Are we getting close to the goal?
  - What if we know something about the search?
  - How should we include that knowledge?
  - In what form should it be expressed to be useful?
- Use a ***heuristic function*** that ***estimates*** how close a state is to the goal
- A heuristic is designed for a particular search problem
  - Problem specific (hence informed)
  - *Example:* Manhattan distance, Euclidean distance for pathing
- ***A heuristic does NOT have to be perfect!***
- Example of informed search strategies:
  1. Greedy best-first search
  2. A* search
  3. IDA*

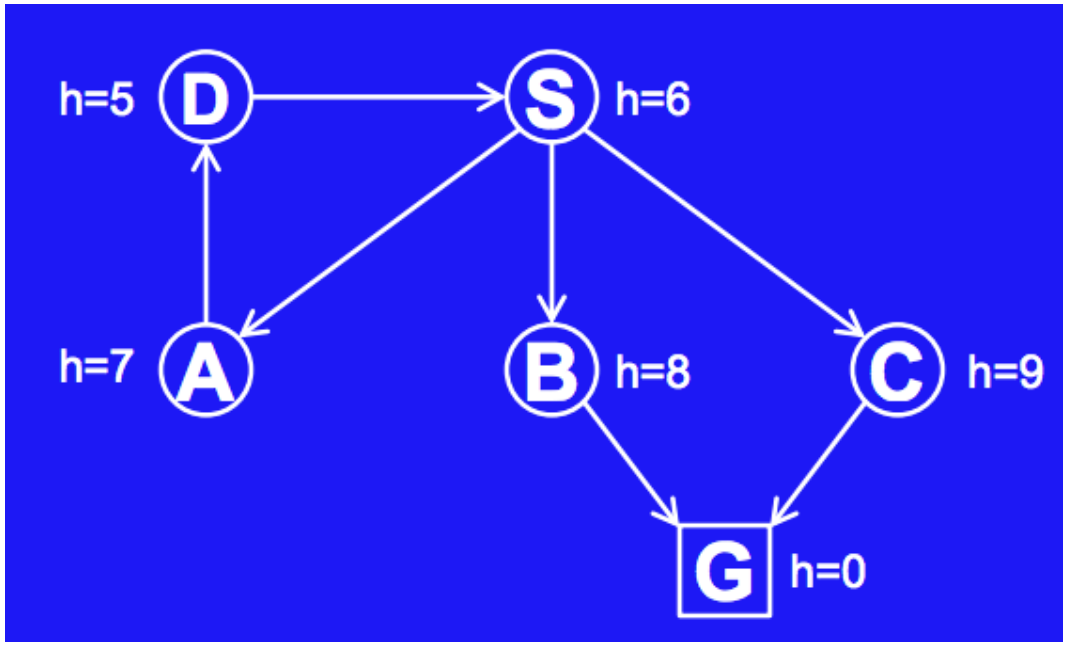
Greedy Best-First Search



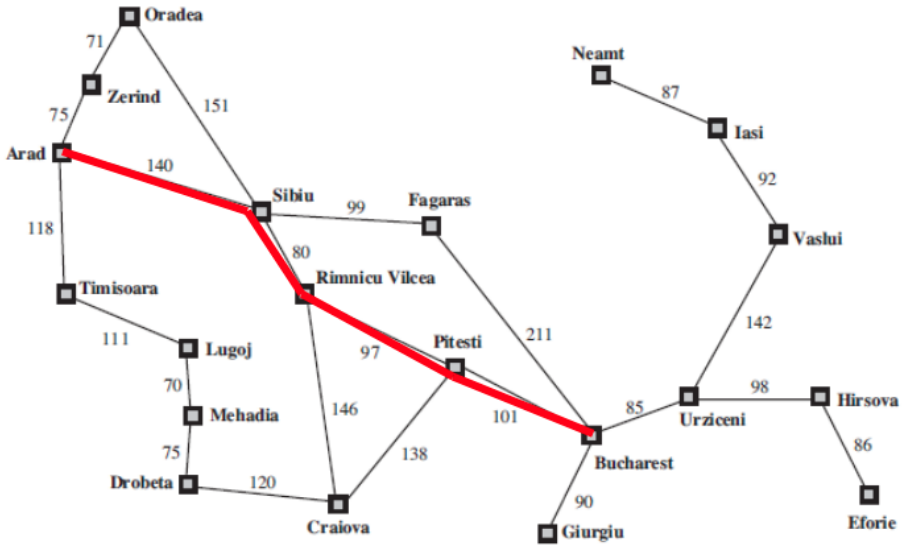
- Evaluation function ***h(n)*** (heuristic)
- ***h(n)* =** estimates the cost from *n* to the closest goal
  - <u>Example:</u> $h_{SLD}(n)$ = straight-line distance from *n* to Bucharest
- Greedy search expands the node that **appears** to be closest to goal
  - Keep nodes on a priority queue sorted by their heuristic value only
    - Priority queue sort function = *h(n)*
  - Compute heuristic function *h(n)* when a node is placed in the frontier
  - Expand the node that appears to be closest to the goal

Order of node expansion: **S A D S A D S A D. . .**
Path found none, *Cost of path found* none

Greedy Search Criteria



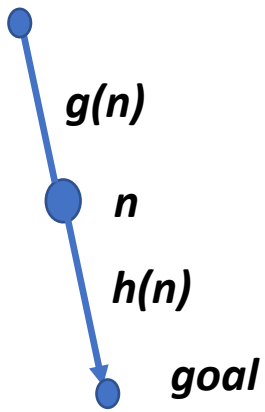
- **Complete** No
  - Tree version can get stuck in loops (try Iasi to Fagaras)
  - Graph version is complete in finite spaces with repeated-state checking
- **Time** $O(b^m)$
  - But a good heuristic can give dramatic improvement
- **Space** $O(b^m)$
  - Keeps all nodes in memory
- **Optimal** No
  - E.g., "Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest" is shorter!



Values of h_{SLD}—straight-line distances to Bucharest.

Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

A* Search



- **Minimize the total estimated solution cost**
- Use a heuristic, but consider the path cost for each node
  - Optimal if heuristic is ***admissible*** (tree search)/***consistent*** (graph search)
- <u>Combines:</u>
  - ✓ ***g(n):*** cost to reach node ***n***
  - ✓ ***h(n):*** cost to get from ***n*** to the goal
  - ✓ ***f(n) = g(n) + h(n)*** ← ***Evaluation Function***

g(n) gives the path cost from the start node to node n*, and*

h(n) is the estimated cost of the cheapest path from n to the goal, so

f(n)* is the estimated cost of the cheapest <u>solution</u> through *n

Admissible Heuristics

- An **admissible** heuristic **<u>never</u>** overestimates the cost to reach the goal, that is, it is **optimistic**
- A heuristic ***h*** is admissible if

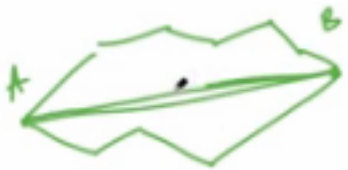
$$\forall\ node\ n, h(n) \leq\ h^*(n)$$

  where $\boldsymbol{h}^*$ is **true** cost to reach the goal from ***n***

- <u>Example</u>

  h_{SLD} (used as a heuristic in the map example) is admissible, because it is by definition the shortest distance between two points

  - Never overestimates the actual road distance



A* Search Criteria

- **Complete** Yes
- **Time** exponential (depends on heuristic)
- **Space** Keeps every node in memory (***the biggest problem***)
- **Optimal** Yes (If ***h(n)*** is admissible)

Local Search

- Search algorithms seen so far are designed to **explore search spaces systematically**
- **Problems:** observable, deterministic, known environments where the solution is a sequence of actions
- Real-World problems are more complex
- When a goal is found, the path to that goal constitutes a solution to the problem. But, depending on the applications, the path may or may not matter
- If the path does not matter/systematic search is not possible, then consider another class of algorithms → **Local Search**

Local Search

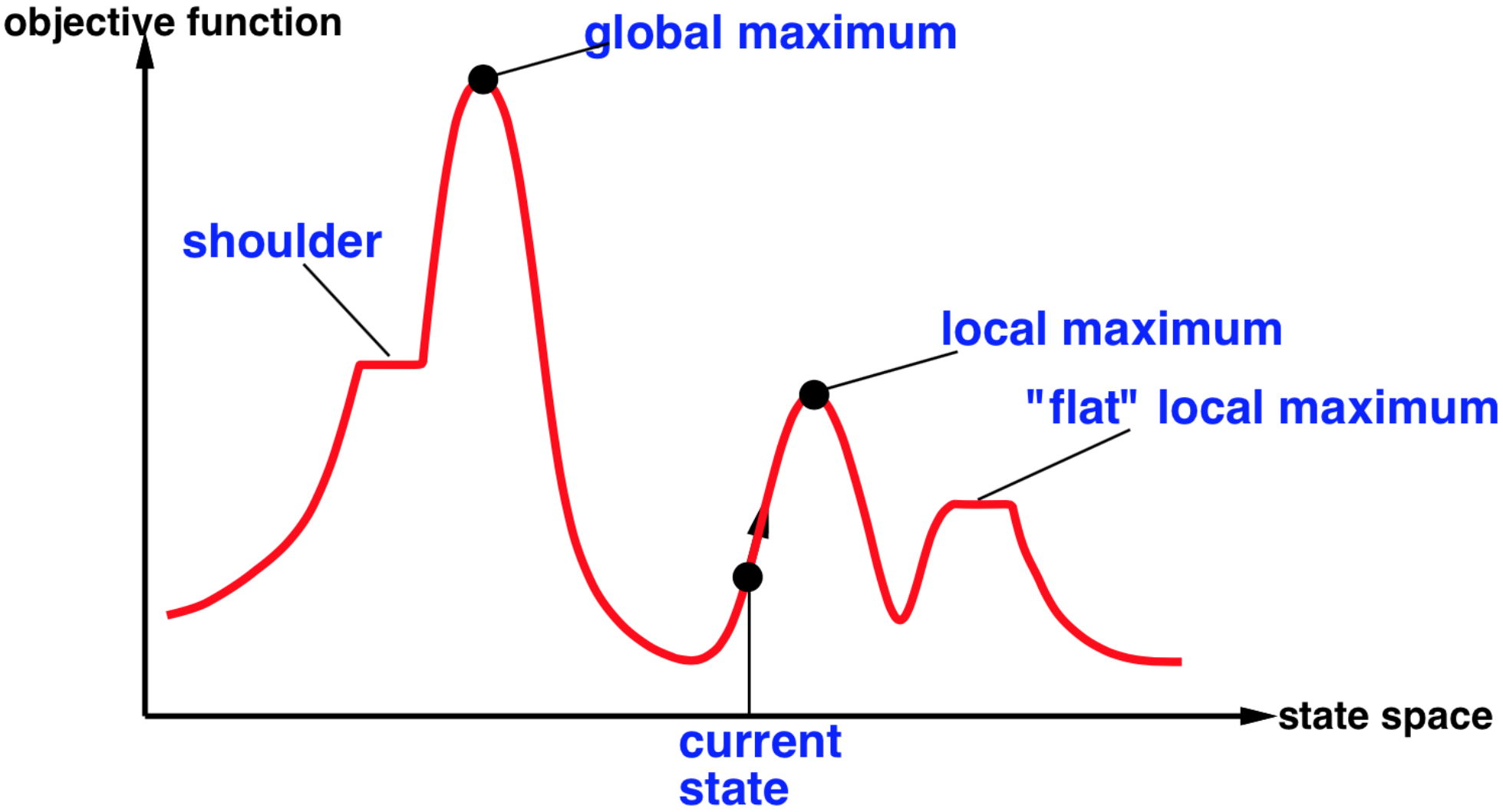
- ***Idea:*** **keep a single "current" state** *(rather than multiple paths)***, and try to improve it**
- Move only to the neighbors of that node
- ***Advantages***
  1. No need to maintain a search tree
  2. Use very little memory
  3. Can often find good enough solutions in continuous or large state spaces

Local Search

Local Search Algorithms

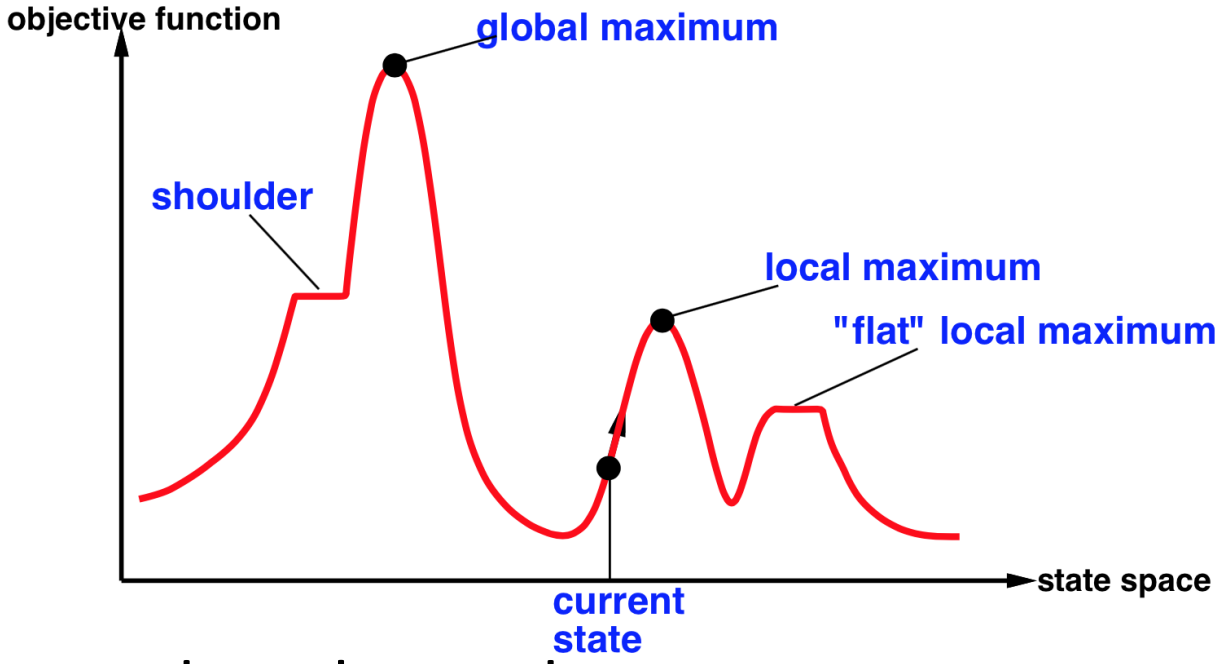
1. **Hill Climbing** (steepest ascent / descent)
2. **Simulated Annealing**: inspired by statistical physics
3. **Local Beam Search**
4. **Genetic Algorithms**: inspired by evolutionary biology

State Space Landscape for Local Search



State Space Landscape

1. Hill Climbing



- Also called **Greedy Local Search**
- Looks only to immediate good neighbors and not beyond
- Search moves uphill:
  - moves in the direction of increasing elevation/value to find the top of the mountain
- Terminates when it reaches a **peak**
- Can terminate with a local maximum, global maximum or can get stuck and no progress is possible
- A **node** is **a state** and **a value**

"Like climbing Everest in thick fog with amnesia"

2. Simulated Annealing

- **Idea: Escape local maxima by allowing "downhill" moves**
  - But make them rarer as time goes on
- Combine hill climbing with a random walk in some way that yields both efficiency and completeness
- In metallurgy, **annealing** is the process used to temper or harden metals and glass by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach a low-energy crystalline state
- Imagine the task of getting a ping-pong ball into the deepest crevice in a bumpy surface
  - If we just let the ball roll, it will come to rest at a local minimum
  - If we shake the surface, we can bounce the ball out of the local minimum.
  - The trick is to shake just hard enough to bounce the ball out of local minima but not hard enough to dislodge it from the global minimum
  - The simulated-annealing solution is to start by shaking hard (i.e., at a high temperature) and then gradually reduce the intensity of the shaking (i.e., lower the temperature)

3. Local Beam Search

- **Local beam search** keeps track of ***k*** states rather than one single state
  - Initially: Begin with ***k*** randomly generated states
  - Next: all the successors of all ***k*** states are generated
  - If any of successors is goal → finished
  - Else **select *k* best successors** from the complete list and repeat

- **Major difference with random-restart search**
  - In a random-restart search, each search process runs independently of the others
  - In a local beam search, useful information is passed among the parallel search threads (states)

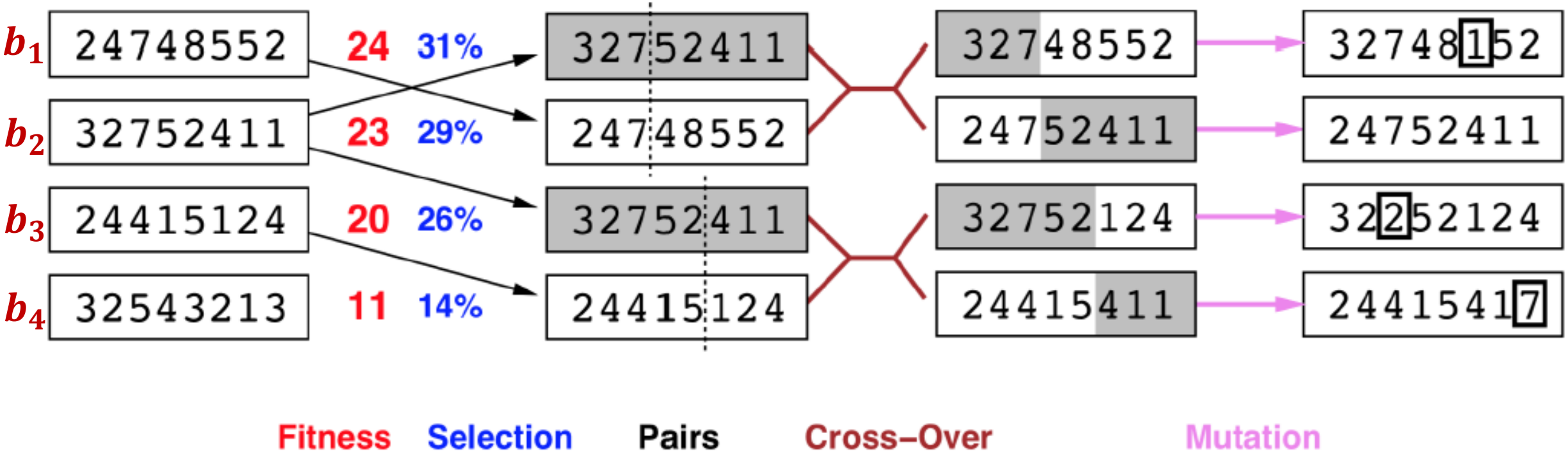
- **Can suffer from lack of diversity among the *k* states** — they can quickly become concentrated in a small region (around the same part) of the state space
  - Help alleviate the problem of the states agglomerating, we can use **Stochastic Beam Search**, where we **choose *k* successors at random**

4. Genetic Algorithms

- **Genetic algorithms (GA)** is **a variant of stochastic beam search**
- Successor states are generated by combining ***two*** parents rather by modifying a single state
- The process is inspired by **natural selection**
  - Same as in stochastic beam search, except that now deal with sexual rather than asexual reproduction
- Starts with ***k* randomly generated states**, called **population**
- Each state is an **individual**
- An individual is usually represented by a **string** of **0**'s and **1**'s, or digits, in general a finite set
- The objective function is called **fitness function**: **better states have high values of fitness function**

4. Genetic Algorithms

Generate successors from pairs of states



Adversarial Search

- **Adversarial Search Problems ≡ Games**
- They occur in ***competitive*** multi-agent environments
  - There is an **opponent** we can't control planning against us!
  - Agent's goals are in conflict!! Each agent tries to optimize a ***utility***
  - ***Zero-sum-game***: One agent's gain in utility, is balanced by the opponent's loss in utility
    - **One function:** ***one agents maximizes it and one minimizes it!***
- We will assume that there are only two agents, the environment is deterministic, and fully observable

Adversarial Search

- ***Game vs. Search:***
  - In games, we look for the optimal solution, which is not a sequence of actions anymore, but a **strategy** ("**policy**") that help us win the game
  - If opponent does ***a***, agent does ***b***, else if opponent does ***c***, agent does ***d***, etc.
- This can be expressed as a set of rules (i.e., implemented with rules)
  - However, tedious and fragile if hard-coded
- ***<u>Good news:</u>***

  Games are modeled as (1) **search problems, algorithms** and

  (2) use **heuristic evaluation functions**

  two main ingredients to model and solve games in AI

Adversarial Search: Minimax

- Two players: ***Max*** and ***Min***
- Players alternate turns
- Max moves first
- Max maximizes results
- Min minimizes the result
- Compute each node's minimax value's the best achievable utility against an optimal adversary
- ***Minimax value*** $\equiv$ ***best achievable payoff against best play***

Adversarial Search: Minimax

- Aims to find the optimal strategy for Max to win:
  - Depth-first search of the game tree
  - An optimal leaf node could appear at any depth of the tree
  - ***Minimax principle:*** compute the utility of being in a state, assuming both players play optimally from there until the end of the game
  - *Idea of the minimax algorithm:* Propagate **minimax** values up the tree once terminal nodes are discovered
    - Go all the way down to the tree and down to the leaves in the tree

The Minimax Algorithm

- Minimax algorithm is designed to determine the optimal strategy for MAX: *Perfect play for deterministic, perfect-information games*
- Idea: choose move to position with ***highest minimax value*** ≡ **best achievable payoff against best play**
- An optimal procedure
  1. Depth-first search of the game tree
  2. Apply utility (payoff) function to each leaf
  3. Back-up values from leaves through branch nodes:
     - A MAX node computes the max of its child values
     - A MIN node computes the min of its child values
  4. At root: Choose move leading to the child of highest value

Adversarial Search: Minimax

- If state is **terminal node**: Value is ***utility(state)***
- If state is **MAX node**: Value is ***highest value of all successor node values*** (children)
- If state is **MIN node**: Value is ***lowest value of all successor node values*** (children)
- ***Recursive Definition:***

For a state s minimax(s) =

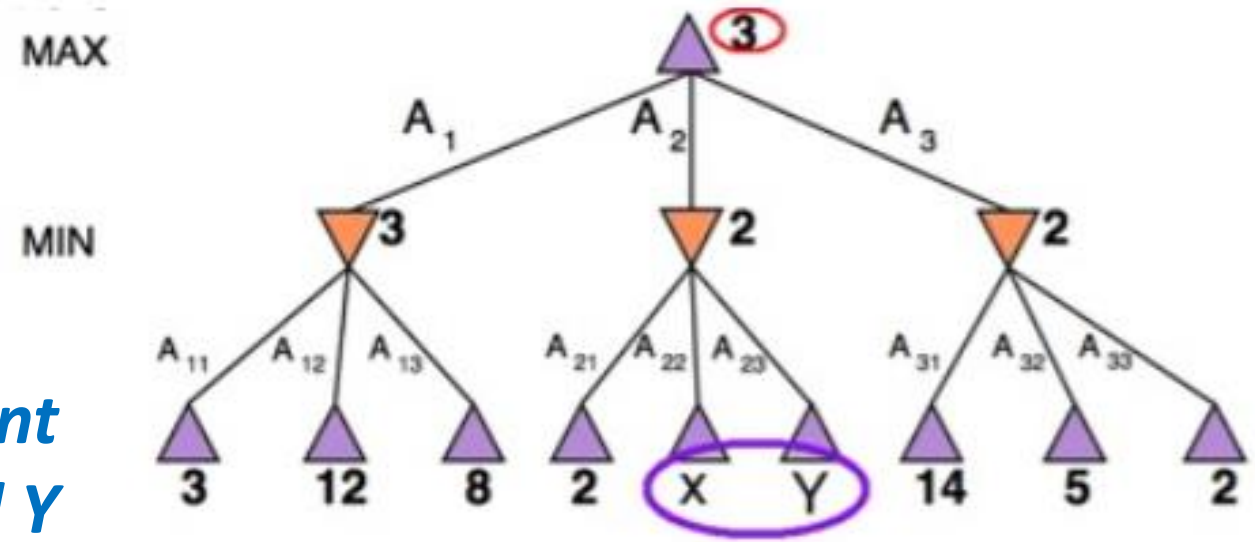
$$\begin{cases} Utility(s) & \text{if Terminal-test(s)} \\ max_{a \in Actions(s)} \text{minimax(Result(s,a))} & \text{if Player(s) = Max} \\ min_{a \in Actions(s)} \text{minimax(Result(s,a))} & \text{if Player(s) = Min} \end{cases}$$

Properties of Minimax

- ***Optimal*** (opponent plays optimally)
- ***Complete*** (finite search tree – possibly huge)
- ***DFS space***: $O(bm)$
- ***DFS time***: $O(b^m)$
  - **Tic-Tac-Toe**
    - * $b \approx 5$ legal moves on average, total of 9 moves (9 plies), so $m = 9$
    - * $5^9 = 1{,}953{,}125$
    - * $9! = 362{,}880$ terminal nodes
  - **Chess**
    - * $b \approx 35$ (average branching factor)
    - * $d \approx 100$ (depth of game tree for a typical game)
    - * $b^d \approx 35^{100} \approx 10^{154}$ nodes
  - **Go** branching factor starts at 361 (19×19 board)

Real Games are Hard!!

$\alpha - \beta$ pruning



Minimax decisions are independent of the values of X and Y

$$Minimax(root) = max(min(3, 12, 8), min(2, X, Y), min(14, 5, 2))$$

$$= max(3, min(2, X, Y), 2)$$

$$= max(3, Z, 2) \qquad \text{where } Z = min(2, X, Y) \leq 2$$

$$= 3$$

$\alpha - \beta$ pruning

- **Strategy:** Just like minimax, it performs a DFS
- **Parameters:** Keep track of two bounds
  - α : **largest** value for **MAX** across seen children (current <u>lower bound</u> on MAX's outcome)
  - β : **lowest** value for **MIN** across seen children (current <u>upper bound</u> on MIN's outcome)
- **Initialization:** $\boldsymbol{\alpha} = -\infty, \quad \boldsymbol{\beta} = \infty$
- **Propagation:** Send $\boldsymbol{\alpha}, \boldsymbol{\beta}$ values ***down*** during the search to be used for pruning
  - Update $\boldsymbol{\alpha}, \boldsymbol{\beta}$ values by ***propagating upwards*** values of terminal nodes
  - Update $\boldsymbol{\alpha}$ only at **MAX nodes** and update $\boldsymbol{\beta}$ only at **MIN nodes**
- **Pruning:** Prune any remaining branches whenever $\boldsymbol{\alpha} \geq \boldsymbol{\beta}$

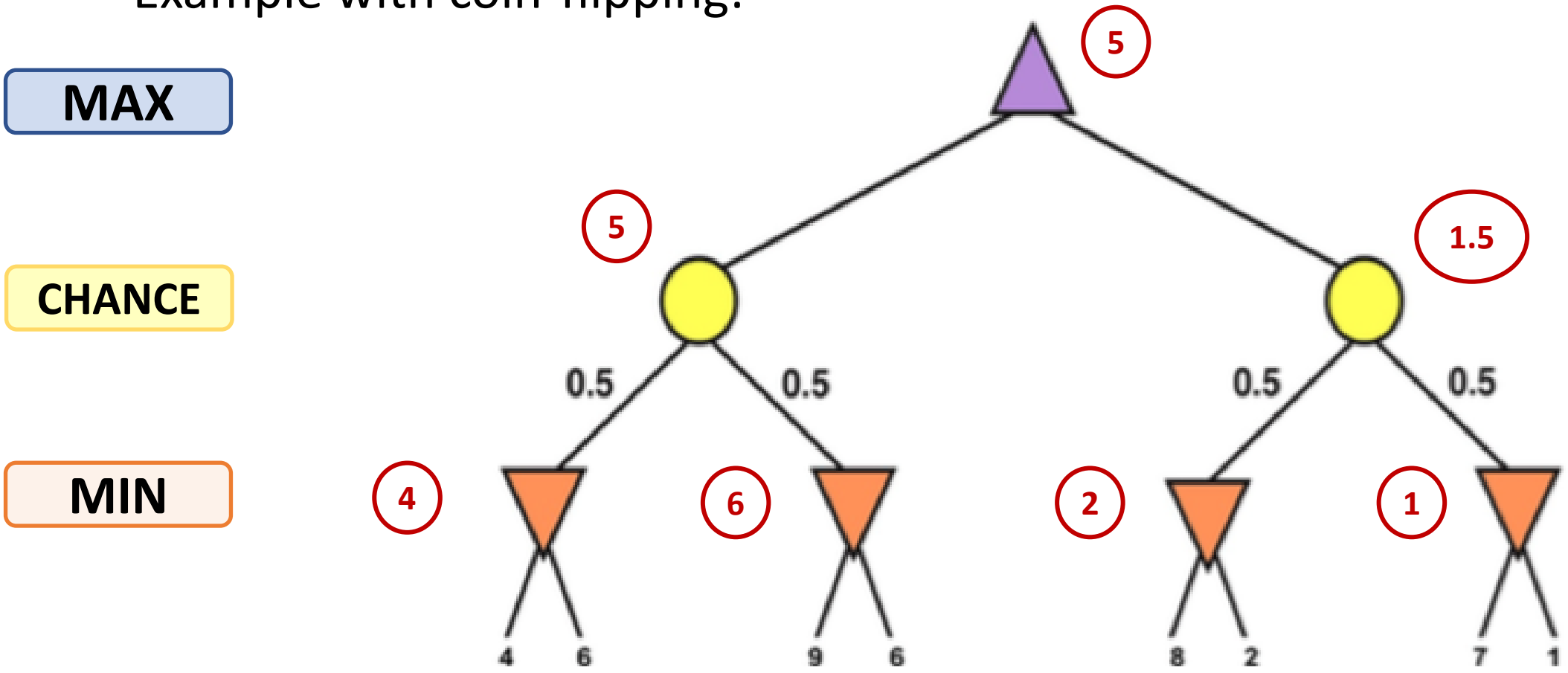
Stochastic Games

Algorithm **Expectiminimax** generalized Minimax to handle chance nodes as follows:

- If state is a **Max** node then
  return the **highest** Expectiminimax-Value of Successors(state)
- If state is a **Min** node then
  return the **lowest** Expectiminimax-Value of Successors(state)
- If state is a **chance** node then
  return **average** of Expectiminimax-Value of Successors(state)

Stochastic Games

Example with coin-flipping:



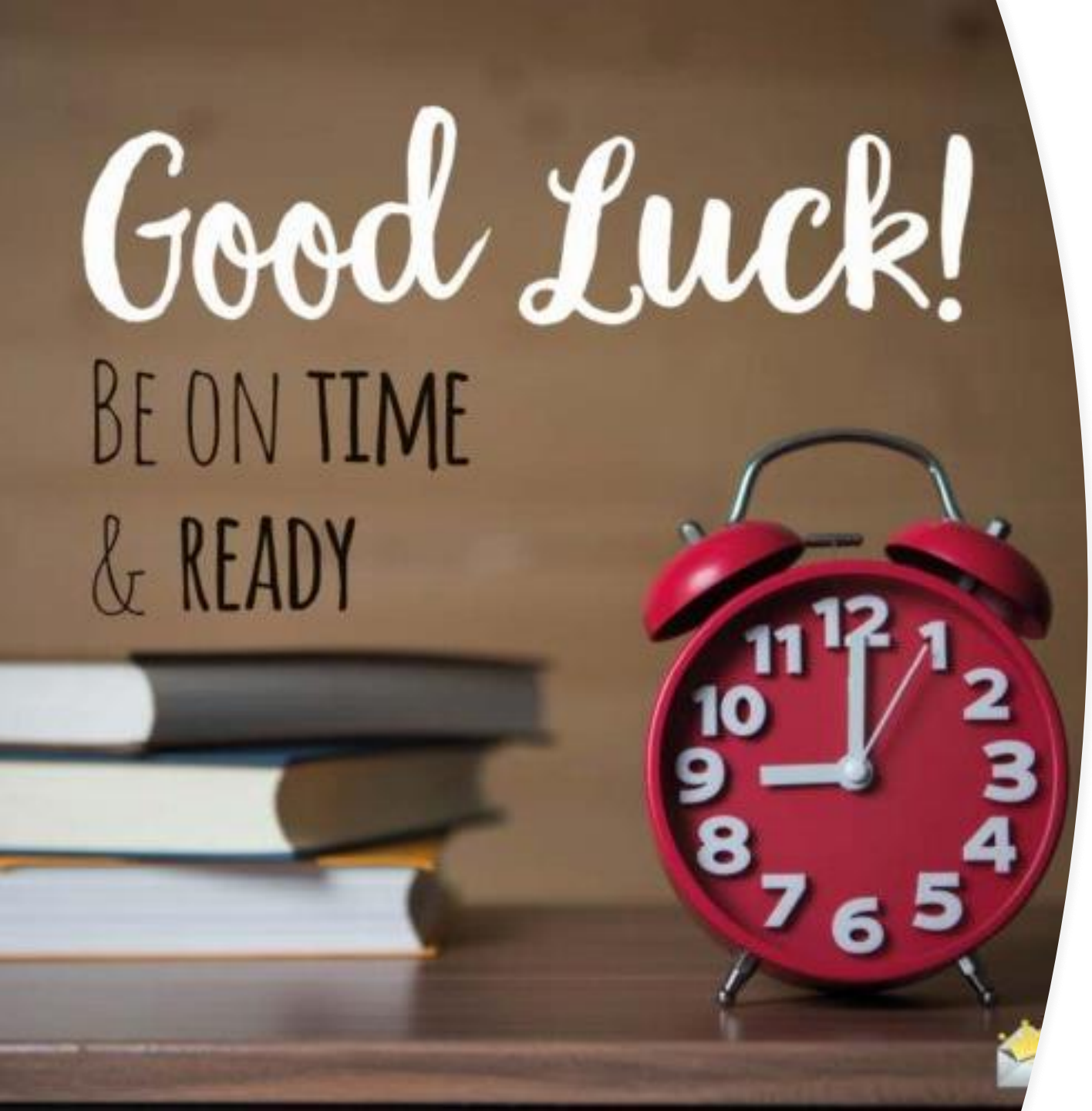
Expectiminimax

For a state s:

Expectiminimax(s) =

$$\begin{cases} Utility(s) & \text{if Terminal-test(s)} \\ max_{a \in Actions(s)} \text{ Expectiminimax(Result(s,a))} & \text{if Player(s) = Max} \\ min_{a \in Actions(s)} \text{ Expectiminimax(Result(s,a))} & \text{if Player(s) = Min} \\ \sum_r P(r) \text{ Expectiminimax(Result(s,r))} & \text{if Player(s) = Chance} \end{cases}$$

Where ***r*** represents all chance events (e.g., dice roll), and ***Result(s,r)*** is the same state as ***s*** with the result of the chance event is ***r***



Exam rules will be announced soon.